

Controle PWM com PIC16F877A

PICGenios + PICSimLab

Engenharia de Computação

- Compreender o funcionamento do módulo PWM¹ do PIC16F877A.
- Configurar o módulo CCP1² em modo PWM.
- Utilizar o Timer2³ para geração do PWM.
- Ler um potenciômetro através do ADC.
- Controlar o duty cycle do PWM.
- Aplicar o PWM no controle de velocidade de um cooler.

¹Pág. 67 do manual do PIC16F877A

²Pág. 63 do manual do PIC16F877A

³Pág. 61 do manual do PIC16F877A

O PWM (Pulse Width Modulation) é amplamente utilizado em:

- Controle de velocidade de motores DC;
- Controle de ventiladores (coolers);
- Controle de brilho de LEDs;
- Fontes chaveadas;
- Inversores de frequência;
- Controle de potência em cargas elétricas.

O PIC16F877A possui dois módulos CCP:

- CCP1
- CCP2

Cada módulo pode operar como:

- 1 Capture
- 2 Compare
- 3 PWM

Neste exemplo será utilizado:

$CCP1 \rightarrow RC2$

Módulo CCP1 em modo PWM

- O **CCP1** é configurado em modo PWM através do registrador **CCP1CON**.
- O Timer2 fornece a base de tempo do PWM.
- O registrador PR2 determina o período do PWM.
- Os registradores **CCPR1L** e **DC1B1:DC1B0** determinam o **duty cycle**.
- A saída PWM é disponibilizada no pino RC2/CCP1.

CCP Mode	Timer Resource
Capture	Timer1
Compare	Timer1
PWM	Timer2

CCP significa:

Capture – Compare – PWM

O módulo CCP1 é um periférico de hardware capaz de operar em três modos distintos:

- 1 Capture (Captura)
- 2 Compare (Comparação)
- 3 PWM (Pulse Width Modulation)

O modo de operação é selecionado pelos bits CCP1M3:CCP1M0 do registrador CCP1CON.

Modos de Operação do CCP1

Modo	Timer	Aplicação
Capture	Timer1	Medição de sinais
Compare	Timer1	Eventos temporizados
PWM	Timer2	Controle de potência

Pino Utilizado

Nos três modos, o módulo CCP1 utiliza o pino:

RC2/CCP1

- Utiliza o Timer1 como base de tempo.
- Detecta eventos no pino RC2/CCP1.
- Ao detectar uma borda, copia o valor do Timer1 para:

CCPR1H:CCPR1L

- Permite medir:
 - frequência;
 - período;
 - largura de pulso;
 - velocidade de rotação.

Evento no RC2 → Captura do Timer1

- Utiliza o Timer1.
- O valor de CCPR1H:CCPR1L é comparado continuamente com o Timer1.
- Quando ocorre igualdade:
 - gera interrupção;
 - altera o estado do pino RC2;
 - pode disparar evento especial.

Timer1 = CCPR1



Interrupção ou ação programada

Modo PWM

- Utiliza o Timer2.
- Gera um sinal PWM no pino RC2/CCP1.
- Resolução de até 10 bits.
- O período é definido por:

PR2

- O duty cycle é definido por:

CCPR1L e CCP1CON<5:4>

Aplicações:

- Controle de motores;
- Controle de cooler;
- Controle de brilho de LEDs;
- Fontes chaveadas.

Por que o PWM utiliza o Timer2?

PR2 → Timer2 → CCP1 (PWM) → RC2

- O registrador PR2 define o período do PWM.
- O Timer2 realiza a contagem necessária para gerar a frequência do PWM.
- O módulo CCP1 compara a contagem do Timer2 com o valor do duty cycle.
- O sinal PWM é disponibilizado no pino RC2/CCP1.

Observação

O Timer2 possui:

- Prescaler;
- Registrador de período (PR2);
- Postscaler.

Esses recursos permitem gerar sinais PWM com frequência e resolução controladas por hardware.

Seleção do Modo no Registrador CCP1CON

CCP1M3:CCP1M0	Modo
0000	CCP desligado
0100 a 0111	Capture
1000 a 1011	Compare
1100	PWM

Importante

Embora o CCP1 possua três modos principais, os modos Capture e Compare apresentam diversos submodos de operação.

Potenciômetro → ADC → PWM → Cooler

- AN0 lê a posição do potenciômetro.
- ADC converte tensão para valor digital.
- Duty cycle é ajustado conforme o ADC.
- PWM controla a velocidade do cooler.

Dispositivo	Pino PIC
Potenciômetro	RA0/AN0
PWM	RC2/CCP1
Cooler	RC2

Frequência PWM usando Timer 2

A frequência é definida por:

$$f_{PWM} = \frac{F_{osc}}{4 \times Prescaler \times (PR2 + 1)}$$

Para:

$$F_{osc} = 20MHz$$

$$Prescaler = 16$$

$$PR2 = 63$$

obtemos:

$$f_{PWM} = 4882Hz$$

Registrador	Função
CCP1CON	Configuração PWM
CCPR1L	Duty Cycle
T2CON	Timer2
PR2	Período PWM
ADCON0	Controle ADC
ADCON1	Configuração ADC

Inicialização do ADC

```
void adc_init(void) {  
    ADCON0 = 0x41;  
    ADCON1 = 0x80;  
}
```

Funções:

- Seleciona AN0.
- Liga o conversor A/D.
- Resultado justificado à direita.

Leitura do ADC

```
uint16_t adc_read(uint8_t canal) {  
    ADCON0 &= 0xC5;    // ADCON0bits.CHS = canal;  
    ADCON0 |= (canal<<3);  
  
    __delay_us(20);  
  
    GO_nDONE = 1;  
  
    while(GO_nDONE);  
  
    return ((ADRESH<<8)+ADRESL);  
}
```

Inicialização do PWM

```
void pwm_init(void) {  
    TRISC2 = 0; //TRISCbits.TRISC2 = 0;  
  
    PR2 = 63;  
  
    CCP1CON = 0b00001100;  
  
    T2CON = 0b00000111;  
  
    CCPR1L = 0;  
  
    TMR2ON = 1;  
}
```

$$CCP1CON = 00001100_2$$

Bits:

Bits	Função
CCP1M3:CCP1M0	Modo PWM
DC1B1:DC1B0	2 bits do duty

Ajuste do Duty Cycle

```
void pwm_set_duty(uint16_t duty) {  
    CCPR1L = duty >> 2;  
  
    CCP1CONbits.DC1B1 = (duty & 0x02)>>1;  
  
    CCP1CONbits.DC1B0 = (duty & 0x01);  
}
```

$$\text{Duty}_{10\text{bits}} = (\text{CCPR1L} : \text{DC1B1} : \text{DC1B0})$$

$$\underbrace{b_9 \ b_8 \ b_7 \ b_6 \ b_5 \ b_4 \ b_3 \ b_2}_{\text{CCPR1L}} \quad \underbrace{b_1 \ b_0}_{\text{DC1B1:DC1B0}}$$

Resolução do PWM

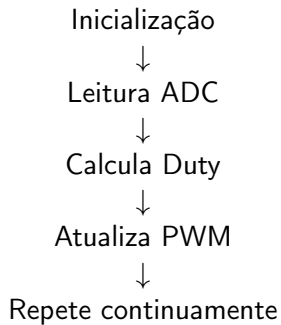
O PWM possui resolução de 10 bits.

$$0 \leq Duty \leq 1023$$

Correspondência:

Valor	Duty
0	0%
512	50%
1023	100%

```
while(1) {  
    adc = adc_read(0);  
  
    duty = adc;  
  
    pwm_set_duty(duty);  
  
    __delay_ms(10);  
}
```



Comportamento Esperado

ADC	Duty	Cooler
0	0%	Parado
256	25%	Baixa velocidade
512	50%	Média velocidade
768	75%	Alta velocidade
1023	100%	Máxima velocidade

Vantagens do PWM

- Alta eficiência energética.
- Baixa dissipação de potência.
- Controle preciso de velocidade.
- Fácil implementação por hardware.
- Não ocupa tempo excessivo da CPU.

- 1 Alterar a frequência PWM para 10 kHz.
- 2 Controlar o brilho de um LED usando PWM.
- 3 Exibir o valor do duty cycle em um LCD.
- 4 Utilizar botões para aumentar e diminuir o duty cycle.
- 5 Implementar controle automático de temperatura usando PWM.